// 2026.03.30

[PROOF OF CONCEPT]

AUTOMATED PENETRATION TESTING FRAMEWORK

GLAZ

Network Reconnaissance // Vulnerability Mapping // Credential Auditing

BASH NMAP HYDRA SEARCHSPLOIT NSE RED TEAM

Denis Albin

albindeniswork@gmail.com

Cybersecurity Student | Red Team Enthusiast

01 // Executive Summary

Glaz is an automated penetration testing framework written entirely in Bash. It provides a complete workflow for network security assessments: from initial target acquisition and scan configuration, through active reconnaissance with Nmap, to credential auditing with Hydra and vulnerability mapping via NSE scripts and Searchsploit. All findings are organized into timestamped reports with multiple output formats.

KEY CAPABILITIES

- Automated tool verification and installation (Nmap, Hydra, Searchsploit, ipcalc, xmlstarlet)
- Interactive TUI for configuring scan parameters: type, aggressiveness, ports, and target
- Dual-mode operation: Basic scan (TCP/UDP + credential audit) and Full scan (+ NSE + vulnerability mapping)
- Systematic brute-force auditing of SSH, FTP, SMB, Telnet, and SMTP with built-in or custom wordlists
- Searchsploit integration for mapping discovered services to known CVEs and exploits
- Structured reporting: log, XML, and auto-generated HTML reports in timestamped directories
- Built-in scan history browser with quick/deep search via hidden signature files
- Optional ZIP compression of all scan artifacts for portability

TOOL STACK

`Bash` `Nmap` `Hydra` `Searchsploit` `xmlstarlet` `ipcalc` `xsltproc`

WORKFLOW



02 // Target Acquisition

The **ip_input** function provides an interactive interface for defining the scope of the penetration test. It prompts for a target IP address and CIDR subnet mask with full input validation. A key feature is auto-detection: Glaz can read the configuration of the **eth0** interface to automatically populate the target IP and CIDR, saving time during engagements. The user also defines the output directory name, with 'Glaz' as the default.

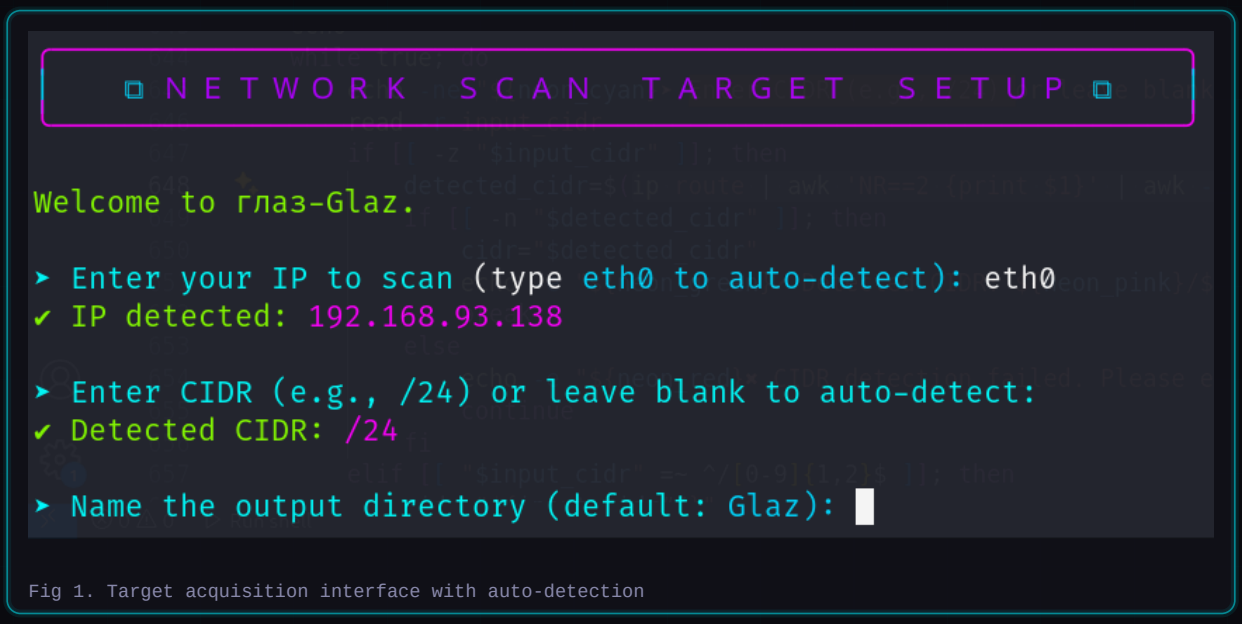


Fig 1. Target acquisition interface with auto-detection

INPUT VALIDATION

Glaz implements comprehensive input validation throughout the entire workflow. IP addresses are validated against proper formatting rules, CIDR masks are checked for valid ranges (0-32), and port specifications are verified to be within the 1-65535 range. Invalid inputs trigger clear error messages with guidance on correct formats, ensuring the scan never proceeds with malformed parameters.

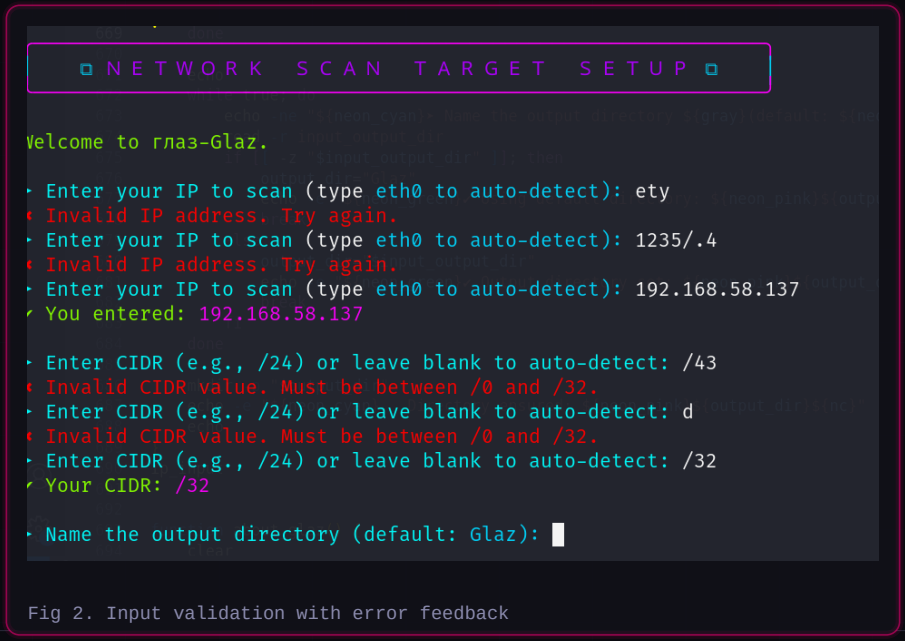


Fig 2. Input validation with error feedback

03 // Scan Configuration

The **scan_core** function presents a detailed interactive TUI menu where the user configures every aspect of the scan. Two primary modes are available:

BASIC SCAN

- TCP/UDP port discovery
- Service version detection
- OS fingerprinting
- Weak credential auditing

FULL SCAN

- Everything in Basic, plus:
- NSE script selection & execution
- Vulnerability mapping (Searchsploit)
- Automated CVE correlation

Additional configurable options include: scan type (SYN Stealth, Connect, ACK, Window), aggressiveness levels (Ghost T1 through Insane T5), UDP scanning toggle, custom port ranges, and output format selection. Every toggle updates the menu display in real-time.



Fig 3. Interactive scan configuration menu

04 // Credential Auditing

When the Brute Force option is enabled, Glaz parses Nmap scan results to identify hosts running common login services. For each discovered service, it launches targeted attacks using **Hydra** to test for weak passwords. Supported protocols include SSH, FTP, Telnet, SMTP, and SMB. The tool uses either its built-in default wordlist or a user-supplied custom list.

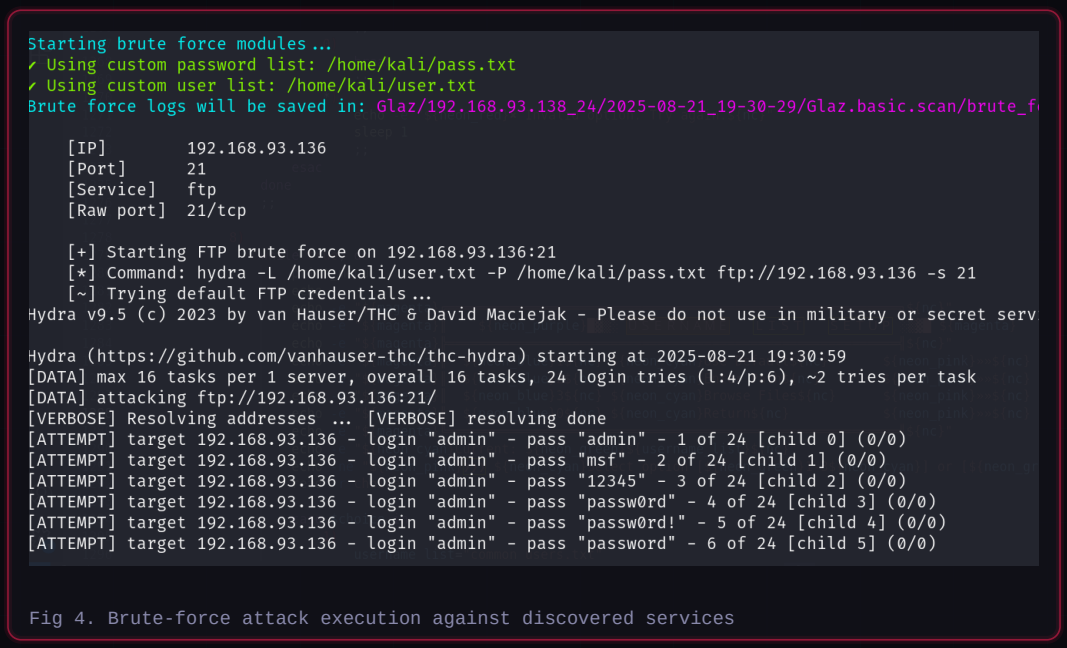


Fig 4. Brute-force attack execution against discovered services

WORDLIST MANAGEMENT

Glaz ships with a comprehensive built-in password and username list totaling **29,766 login attempts**. For more targeted engagements, users can supply their own custom wordlists via an integrated file browser with **cd** navigation support. The file browser lets the user navigate the filesystem to select custom password and username lists.

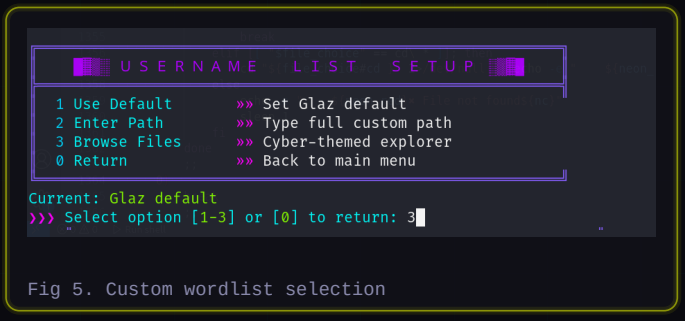


Fig 5. Custom wordlist selection

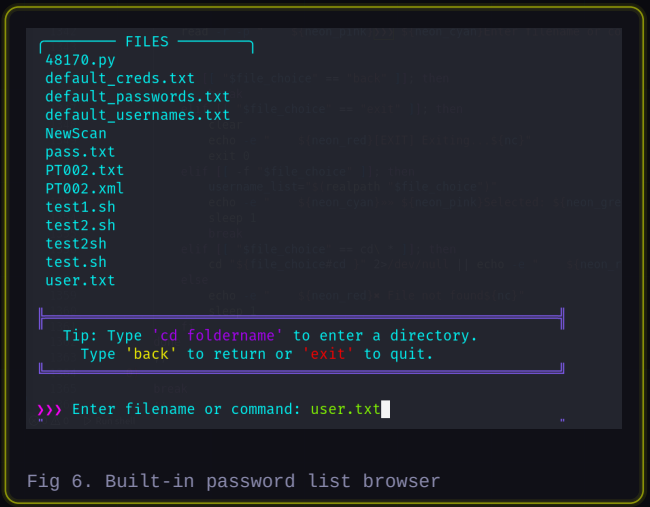


Fig 6. Built-in password list browser

05 // Vulnerability Mapping

Available exclusively in Full Scan mode, vulnerability mapping is Glaz's most powerful feature. The script uses **xmlstarlet** to parse Nmap XML output, extracting each service's product name and version correlated with its host IP. It then queries **Searchsploit** (updated automatically before each run via **searchsploit -u**) to find known exploits and CVEs for every discovered service.

SMART DEPENDENCY MANAGEMENT

If Service Detection is disabled, Vuln Map auto-disables. Enabling Vuln Map while Service Detection is off will automatically re-enable it.

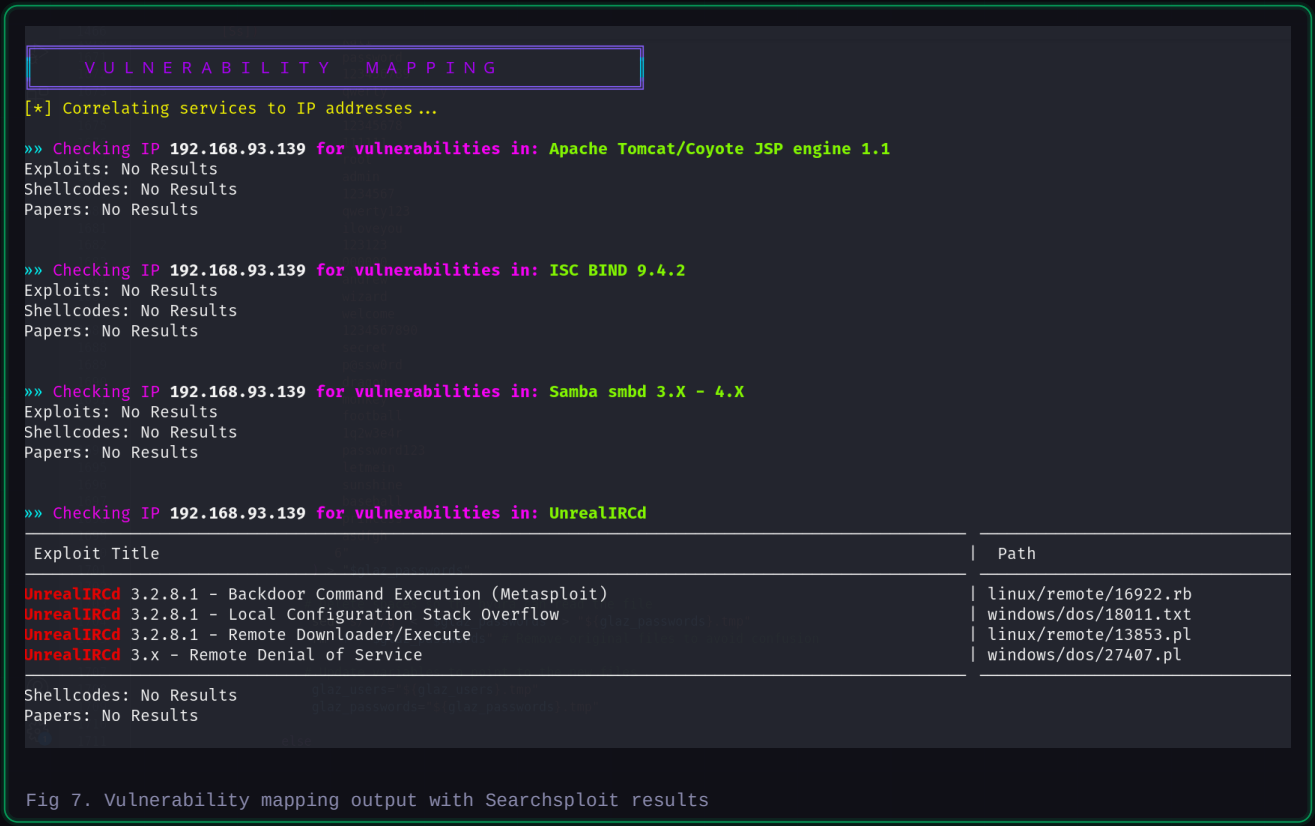


Fig 7. Vulnerability mapping output with Searchsploit results

06 // NSE Script Browser

For Full Scan mode, Glaz includes a built-in Nmap Scripting Engine (NSE) browser that lets the user explore and select from all available Nmap scripts. The browser includes search functionality to quickly find specific scripts by keyword, and selected scripts are attached to the scan command automatically. This gives the operator granular control over which vulnerability checks to run.

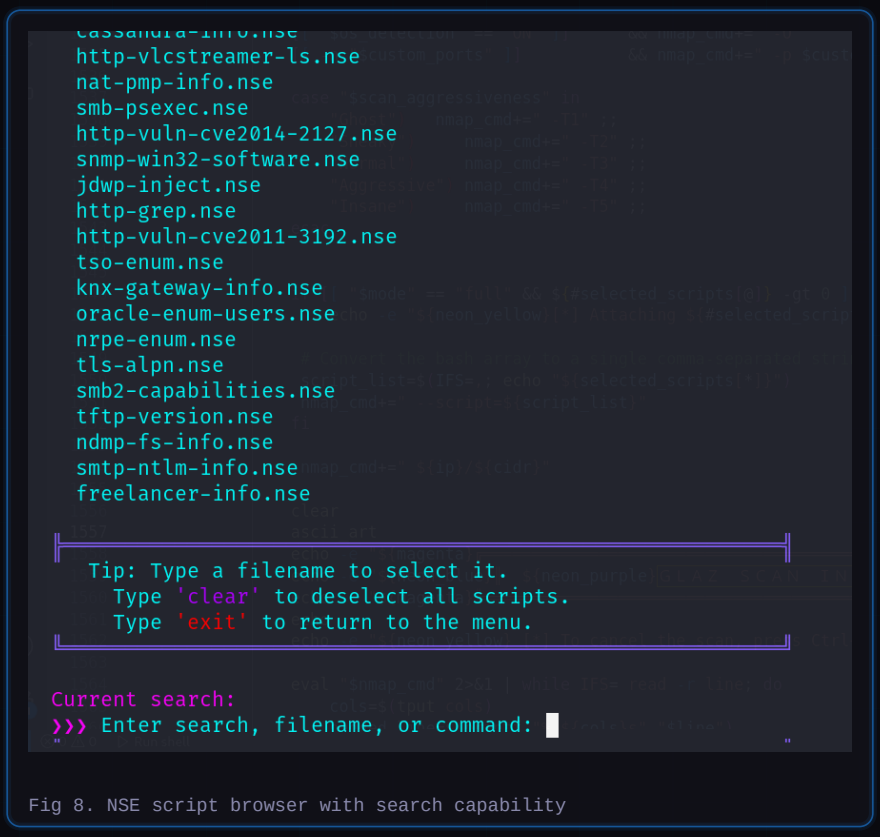


Fig 8. NSE script browser with search capability

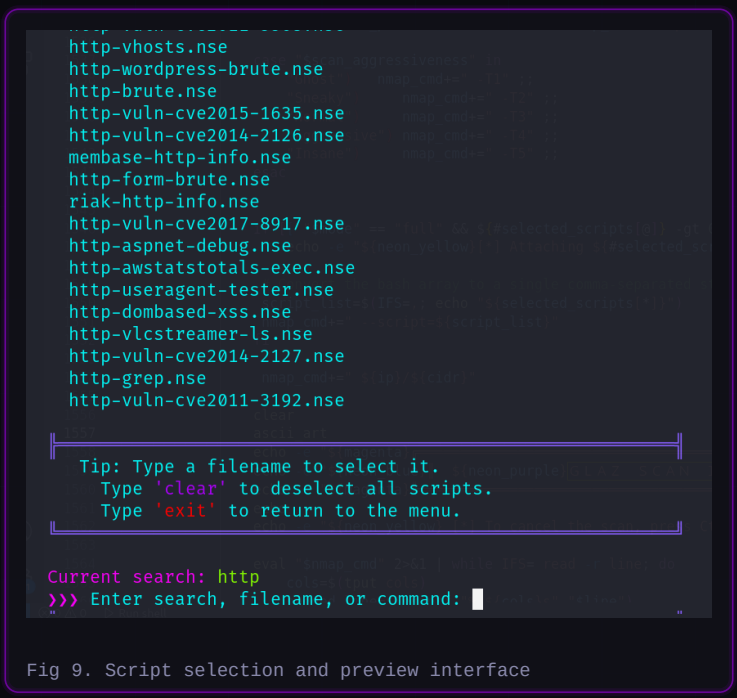


Fig 9. Script selection and preview interface

07 // Live Monitoring & Reporting

Throughout each stage of operation, Glaz provides a **real-time live feed** of all actions and findings. As it runs Nmap, Hydra, or Searchsploit, output is streamed directly to the terminal. A unique feature is the single-line live output: Nmap progress is rendered on a single updating line rather than scrolling, keeping the terminal clean and readable while providing constant feedback.

```
»» Checking IP 192.168.93.139 for vulnerabilities in: Samba smbd 3.X - 4.X
Exploits: No Results
Shellcodes: No Results
Papers: No Results


»» Checking IP 192.168.93.139 for vulnerabilities in: UnrealIRCd
 Exploit Title                                                    |  Path
UnrealIRCd 3.2.8.1 - Backdoor Command Execution (Metasploit)      | linux/remote/16922.rb
UnrealIRCd 3.2.8.1 - Local Configuration Stack Overflow           | windows/dos/18011.txt
UnrealIRCd 3.2.8.1 - Remote Downloader/Execute                    | linux/remote/13853.pl
UnrealIRCd 3.x - Remote Denial of Service                         | windows/dos/27407.pl
```

Fig 10. Live scan progress with single-line updating output

STRUCTURED RESULTS

Upon completion, Glaz generates a comprehensive results summary. Reports are saved in multiple formats: raw logs (.log), XML for machine parsing, and auto-generated HTML reports via **xsltproc**. All artifacts are organized into timestamped directories, making it easy to track and compare multiple engagements over time.

LOG XML HTML ZIP

All output formats are generated automatically and stored in timestamped directories

```
Brute force modules have completed.
Successful logins found:
1
```

Fig 11. Exploit count summary after vulnerability mapping

08 // Scan History & Search

Glaz provides a powerful search system for locating and reviewing past scans. Each completed scan creates a hidden **.DATA** file containing a Base64-encoded signature (*"This is a Glaz mark"*). The search function scans the filesystem for these markers, offering both **Quick Scan** (searches common locations) and **Deep Scan** (full filesystem search) modes.

Results are presented in a clean, date-sorted list. Users can select any scan to dive into its detailed results: viewing vulnerability reports, checking brute-force logs, or opening the full HTML report directly from the terminal interface.

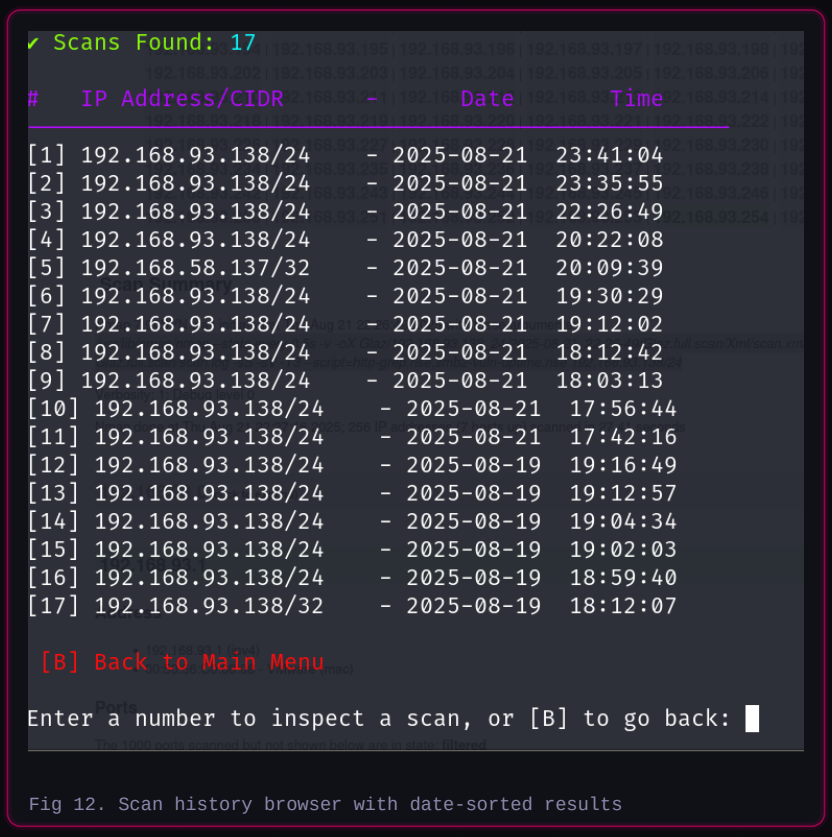


Fig 12. Scan history browser with date-sorted results

ZIP EXPORT

For portability, Glaz can automatically compress all scan artifacts into a single ZIP file. This optional feature bundles Nmap logs, HTML reports, and all generated data into one portable archive.

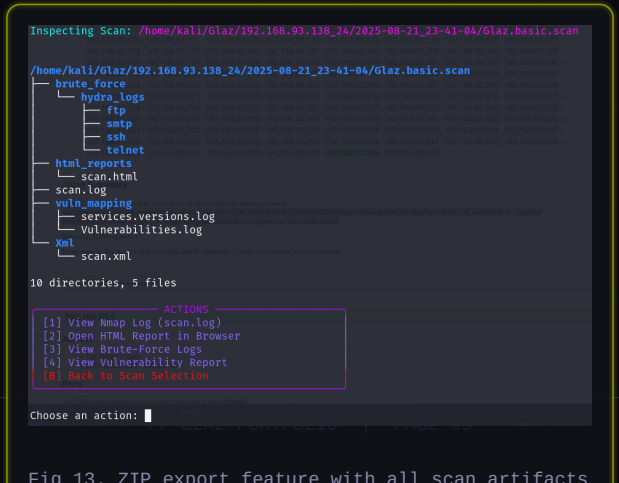


Fig 13. ZIP export feature with all scan artifacts

09 // UI/UX & Design

Glaz is designed with a strong cyberpunk aesthetic that makes it visually distinct from typical security tools. Key design decisions include randomized ASCII art banners displayed on launch (with a different banner and color each time), a neon color palette throughout the interface, and responsive layout that adapts to terminal dimensions using **tput cols** and **tput lines**.

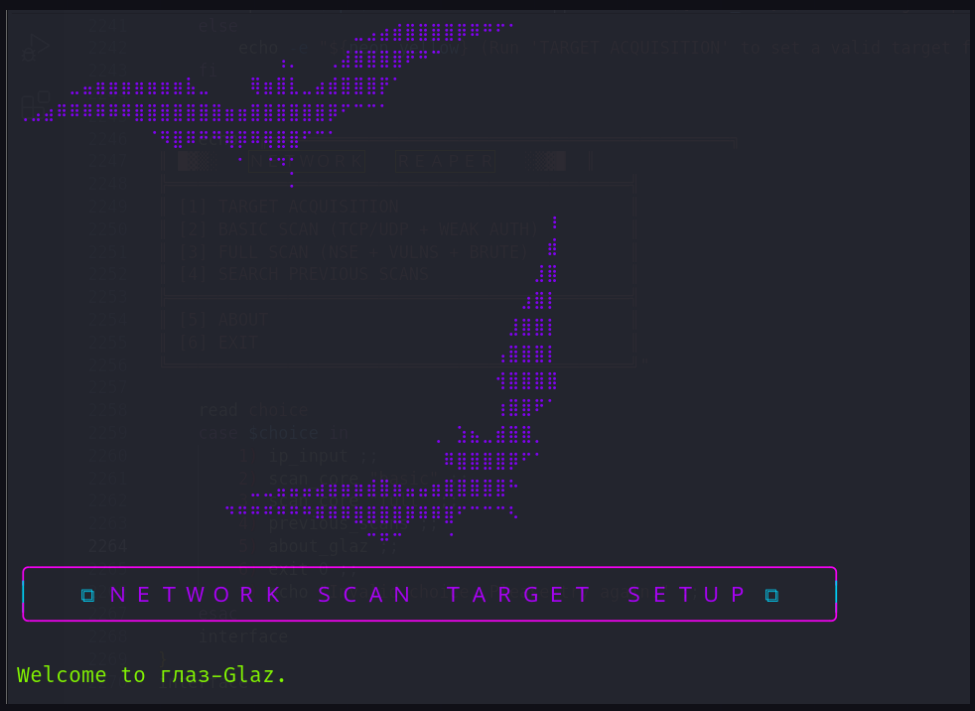


Fig 14. Randomized ASCII art banner with neon color theme

DESIGN HIGHLIGHTS

- ASCII art only renders when terminal is 70+ cols wide and 20+ lines tall, preventing layout breakage
- Live Nmap output updates on a single line in real-time, keeping the terminal clean
- Neon color coding differentiates errors (red), success (green), prompts (cyan), and warnings (yellow)
- Unicode box-drawing characters create structured, professional menu layouts

10 // Technical Deep Dive

SIGNAL HANDLING & GRACEFUL INTERRUPTS

Glaz implements custom **trap** handlers for INT and TERM signals. During active scans, pressing Ctrl+C triggers a clean interruption: the cursor is restored, traps are reset, and control returns to the menu loop via a flag-based mechanism rather than abrupt termination. This ensures the user never loses their session state mid-engagement.

DYNAMIC COMMAND CONSTRUCTION

The Nmap command is built dynamically based on user configuration. Each toggle (UDP, OS detection, service versioning) appends the corresponding flag. Scan type maps to specific Nmap switches (e.g., SYN Stealth maps to -sS, Connect to -sT). Aggressiveness levels map to timing templates (T1 through T5). Custom ports and NSE scripts are appended with proper quoting to prevent injection.

XML PARSING & SERVICE CORRELATION

Vulnerability mapping relies on **xmlstarlet** to parse Nmap's XML output. The script extracts service product names and versions, correlating each with its host IP address. This structured data is then fed to **Searchsploit** to query the Exploit-DB database for matching vulnerabilities. Before each query, Searchsploit's database is updated via **searchsploit -u** to ensure the latest exploit data is available.

AUTOMATED ENVIRONMENT SETUP

On launch, Glaz runs a **tool_check** function that verifies the presence of all required tools: nmap, xmlstarlet, searchsploit, medusa, hydra, and ipcalc. While medusa is installed for future expansion, brute-force operations currently use Hydra exclusively. Any missing tool is automatically installed via apt-get with proper error handling. If installation fails, the user receives a clear manual installation command before the script exits gracefully.

SCAN SIGNATURE SYSTEM

Each completed scan generates a hidden **.DATA** file containing a Base64-encoded marker. This signature system enables the scan history browser to locate past Glaz scans anywhere on the filesystem without requiring a centralized database. The search algorithm supports both quick (targeted directories) and deep (full filesystem traversal) modes.

AUTOMATED HTML REPORT GENERATION

After each scan completes, Glaz uses **xsltproc** to transform Nmap's XML output into a styled HTML report. These reports are stored alongside the raw logs in the timestamped output directory, providing a browser-friendly view of all scan results that can be shared with stakeholders or included in formal penetration testing reports.

GLAZ

AUTOMATED PENETRATION TESTING FRAMEWORK

2,200+	29,766	5	4
Lines of Code	Built-in Login Attempts	Supported Protocols	Output Formats

Denis Albin

albindeniswork@gmail.com

// END OF DOCUMENT

Built with passion for cybersecurity